

**INFORMAÇÕES**

| MÓDULO                                              | AULA                          | INSTRUTOR             |
|-----------------------------------------------------|-------------------------------|-----------------------|
| 09    Introdução ao React para desenvolvedores .NET | 01 - Introdução ao TypeScript | Pedro Henrique Amadio |

**CONTEÚDO****1. O que é TypeScript?**

O TypeScript é uma linguagem criada para tornar o desenvolvimento com JavaScript mais seguro, mais organizado e mais previsível. De forma simples, podemos dizer que o TypeScript é uma camada acima do JavaScript. Ele aproveita toda a base do JavaScript, mas adiciona recursos extras que ajudam o desenvolvedor a escrever código com menos erros. Se no JavaScript nós temos muita liberdade, no TypeScript nós continuamos com essa liberdade, mas com mais controle.

O grande diferencial do TypeScript é a **tipagem estática**. Isso significa que podemos informar com mais clareza que tipo de dado uma variável deve armazenar, que tipo de valor uma função espera receber e que tipo de resultado ela deve retornar. Na prática, isso ajuda muito porque vários erros passam a ser encontrados **antes mesmo de o programa rodar**.

O TypeScript é muito utilizado no mercado atual, principalmente em projetos com: Frontend moderno;

- React;
- Node.js;
- APIs;
- sistemas maiores que precisam de manutenção;
- equipes com vários desenvolvedores trabalhando no mesmo projeto.

Em resumo:

**JavaScript** é a base.

**TypeScript** é o JavaScript com recursos extras para desenvolvimento mais profissional.

**2. Relembrando o JavaScript para entender o TypeScript**

Antes de entender o **TypeScript**, vale relembrar uma característica importante do **JavaScript**: **ele é uma linguagem dinâmica**. Esse comportamento já aparece nas aulas-base do módulo de JavaScript, quando uma variável pode mudar de tipo durante a execução.

Veja este exemplo em JavaScript:

```
let valor = 10;  
valor = "dez";  
console.log(valor);
```

Esse código funciona.

### Por quê?

Porque no JavaScript o tipo da variável não precisa ser declarado antes. O motor da linguagem interpreta o tipo do valor em tempo de execução. Isso deixa o JavaScript muito flexível, mas também pode gerar situações perigosas, principalmente quando o projeto começa a crescer.

Por exemplo:

- uma função espera um número, mas recebe uma string;
  - um objeto chega incompleto;
  - alguém altera uma variável sem perceber o impacto;
  - um erro só aparece quando o sistema já está rodando.
- É justamente aí que o TypeScript entra.

## 3. O que muda do JavaScript para o TypeScript?

No TypeScript, podemos declarar o tipo da variável explicitamente.

Exemplo:

```
let valor: number = 10;  
valor = "dez";  
console.log(valor);
```

Nesse caso, o editor e o compilador apontam erro.

Isso acontece porque a variável foi definida como number, e depois tentamos colocar nela uma string.

Então aqui temos uma diferença importante:

- **JavaScript**  
É mais flexível.  
Permite mudanças de tipo com facilidade.
- **TypeScript**  
É mais rigoroso.  
Ajuda a impedir trocas de tipo que possam causar problemas.  
Isso não significa que o TypeScript “engessa” o desenvolvimento.  
Na verdade, ele orienta melhor o programador.

Em resumo:

O JavaScript deixa você fazer quase tudo.

O TypeScript ajuda você a fazer do jeito certo.

## 4. O TypeScript substitui o JavaScript?

Não.

Esse ponto é muito importante.

O TypeScript não substitui o JavaScript. Ele funciona como uma linguagem que é escrita pelo desenvolvedor, mas que depois será transformada em JavaScript.

Ou seja:

1. Nós escrevemos o código em TypeScript;
2. o TypeScript analisa esse código;
3. verifica tipos e possíveis erros;
4. e então gera código JavaScript.

Quem realmente será executado pelo navegador ou pelo ambiente de execução será o JavaScript gerado.

Então podemos dizer:

O navegador não executa TypeScript diretamente. Ele executa JavaScript gerado a partir do TypeScript. Esse processo de transformação é chamado de **compilação**.

## 5. Tipagem estática: a principal ideia do TypeScript

Ela serve para dizer com clareza qual tipo de dado esperamos usar.

Os tipos mais comuns no TypeScript são:

- string
- number
- boolean

Exemplo:

```
let nome: string = "Pedro";  
let idade: number = 25;  
let desenvolvedor: boolean = true;
```

Nesse exemplo:

- nome deve armazenar texto;
- idade deve armazenar número;
- desenvolvedor deve armazenar verdadeiro ou falso.

Se tentarmos fazer algo incorreto, o TypeScript já avisa.

Exemplo:

```
let idade: number = 25;  
idade = "vinte e cinco";
```

Aqui teremos erro, porque "vinte e cinco" é texto, e não número. Isso ajuda muito a evitar bugs.

## 6. Tipos básicos no TypeScript

Vamos ver alguns exemplos simples.

- **String**

```
let linguagem: string = "TypeScript";
```

Representa textos.

- **Number**

```
let versao: number = 5;
```

Representa números inteiros ou decimais.

- **Boolean**

```
let projetoAtivo: boolean = true;
```

Representa verdadeiro ou falso.

- **Arrays**

Também podemos tipar arrays.

```
let tecnologias: string[] = ["HTML", "CSS", "JavaScript", "TypeScript"];
```

Nesse caso, o array aceita apenas strings.

Se tentarmos colocar um número dentro dele, teremos erro.

- **Objetos**

```
let aluno: { nome: string; idade: number } = {  
  nome: "Lucas",  
  idade: 22  
};
```

Aqui o TypeScript também controla a estrutura do objeto.

## 7. O que é o tsconfig.json?

O tsconfig.json é um dos arquivos mais importantes de um projeto TypeScript. Ele funciona como o arquivo de configuração principal da linguagem. Podemos pensar nele como o “painel de controle” do TypeScript.

É nesse arquivo que definimos:

- para qual versão de JavaScript o código será compilado;
- o sistema de módulos;
- o nível de rigidez da verificação;
- comportamento geral do compilador.

Exemplo simplificado:

```
{  
  "compilerOptions": {  
    "target": "ES2020",  
    "module": "ESNext",  
    "strict": true  
  }  
}
```

Esse pequeno trecho já nos mostra três ideias muito importantes, mas raramente precisaremos alterar essas configurações pois é tudo configurado automaticamente.

## 8. Hands-ON: Criando nosso primeiro projeto TypeScript puro (Nosso primeiro Hello World)

Agora vamos fazer nosso primeiro projeto de forma simples, sem framework e sem ferramentas extras mais complexas.

A ideia aqui é focar apenas no funcionamento básico do TypeScript.

## 1. Estruturando nosso projeto

- **Passo 1 - Criar uma pasta para o projeto**

Primeiro, criamos uma pasta para nosso projeto.

Exemplo:

```
mkdir meu-primeiro-typescript
cd meu-primeiro-typescript
```

- **Passo 2 - Inicializar o projeto com npm**

Agora vamos inicializar um projeto Node simples para criar o package.json.

```
npm init -y
```

Esse comando cria um arquivo de configuração do projeto.

- **Passo 3 - Instalar o TypeScript**

Agora vamos instalar o TypeScript como dependência de desenvolvimento.

```
npm install -D typescript
```

Com isso, teremos acesso ao compilador do TypeScript.

- **Passo 4 - Criar o arquivo de configuração**

Agora vamos gerar o tsconfig.json.

```
npx tsc --init
```

Esse comando cria automaticamente o arquivo de configuração do TypeScript.

- **Passo 5 - Criar a estrutura do projeto**

Podemos criar uma pasta chamada src e dentro dela um arquivo chamado index.ts.

Exemplo:

```
mkdir src
```

Depois, criar o arquivo:

```
src/index.ts
```

- **Passo 6 - Instalando o @types do node**

Precisaremos instalar a definição(types) do node para podermos rodar tudo certinho posteriormente.

Para isso executaremos

```
npm install -D @types/node
```

Onde -D significa que o pacote (package) será usado somente para o desenvolvimento.

- **Passo 7 - Configurando o nosso package.json com os arquivos de saída corretos**

O package.json deve parecer-se com algo do tipo

```
{
  "name": "projeto-aula-ts",
  "version": "1.0.0",
  "main": "dist/index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "devDependencies": {
    "@types/node": "^25.4.0",
    "typescript": "^5.9.3"
  }
}
```

- **Passo 8 - Configurando o tsconfig.json**

Vamos configurar o nosso tsconfig para o nosso projeto

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "CommonJS",
    "rootDir": "./src",
    "outDir": "./dist",
    "strict": true,
    "lib": ["ES2020"],
    "types": ["node"]
  },
  "include": ["src"]
}
```

## 2. Criando lógica Hello World

Dentro do arquivo index.ts, podemos escrever:

```
const mensagem: string = "Hello World com TypeScript!";
console.log(mensagem);
```

Esse será nosso primeiro código em TypeScript.

Aqui já conseguimos observar:

- declaração com const;
- definição explícita do tipo string;
- uso do console.log() para exibir a mensagem.

## 3. Compilando o código TypeScript

Agora precisamos transformar o arquivo .ts em .js.

Para isso, usamos o compilador do TypeScript:

`npx tsc`

Ao executar esse comando, o TypeScript compila o projeto e gera os arquivos JavaScript correspondentes.

Dependendo da configuração do tsconfig.json, o arquivo gerado poderá ir para uma pasta como dist.

Perceba que o JavaScript gerado é muito parecido com o código original, mas sem as anotações de tipo.

Isso acontece porque os tipos existem principalmente para ajudar durante o desenvolvimento.

#### 4. Executando o JavaScript gerado

Depois de compilar, podemos executar o arquivo JavaScript com Node.js.

Exemplo:

```
node dist/index.js
```

E no terminal veremos a mensagem:

```
Hello World com TypeScript!
```

#### 5. Exemplo um pouco mais completo

Agora podemos fazer um exemplo ligeiramente mais elaborado:

```
const nome: string = "Maria";
const idade: number = 22;
const estudando: boolean = true;

console.log(`Nome: ${nome}`);
console.log(`Idade: ${idade}`);
console.log(`Estudando: ${estudando}`);
```

Também podemos ver o que acontece se houver erro:

```
const idade: number = 22;
const aluno: string = "Carlos";
```

E depois testar algo incorreto:

```
const idade: number = "vinte e dois";
```

O TypeScript apontará o problema antes da execução.

## 9. Exercícios de fixação

### 1. O que é o TypeScript?

- ☐ Um banco de dados moderno
- ☐ Uma extensão do JavaScript com recursos como tipagem estática
- ☐ Um editor de código
- ☐ Uma biblioteca de CSS

### 2. O navegador executa TypeScript diretamente?

- ☐ Sim
- ☐ Não, ele executa JavaScript gerado a partir do TypeScript
- ☐ Apenas no Chrome
- ☐ Apenas com extensão instalada

### 3. O que significa tipagem estática?

- ☐ O código nunca muda
- ☐ Os tipos dos dados podem ser definidos e verificados antes da execução
- ☐ O programa não usa variáveis
- ☐ O sistema funciona sem compilação

**4. Qual comando instala o TypeScript no projeto?**

- ☐ npm install express
- ☐ npm install -D typescript
- ☐ npm install vite
- ☐ npm install browser

**5. Qual comando cria o arquivo tsconfig.json?**

- ☐ npx start
- ☐ node init
- ☐ npx tsc --init
- ☐ npm config create

**6. Qual comando compila o projeto TypeScript?**

- ☐ node index.ts
- ☐ npx tsc
- ☐ npm browser
- ☐ npm html

**7. O que há de errado no código abaixo?**

```
let idade: number = "vinte";
```

- ☐ Nada, está correto
- ☐ A variável deveria ser const
- ☐ Uma string foi atribuída a uma variável do tipo number
- ☐ O TypeScript não aceita let

**10. Desafio para casa**

Criar um projeto TypeScript simples contendo um arquivo index.ts com:

- uma variável nome do tipo string;
- uma variável idade do tipo number;
- uma variável gostaDeProgramacao do tipo boolean;
- três console.log() exibindo essas informações;
- compilação do arquivo usando npx tsc;
- execução do JavaScript gerado com node.

• **Exemplo (Tá bom vai, essa vai ser 0800)**

```
const nome: string = "Ana";
const idade: number = 19;
const gostaDeProgramacao: boolean = true;

console.log(`Nome: ${nome}`);
console.log(`Idade: ${idade}`);
console.log(`Gosta de programação: ${gostaDeProgramacao}`);
```